Lab 2 Build Guide

Lab 2 Build Guide: Pre-Change Validation Framework

This guide walks through the four-layer validation framework built on top of the Lab 1 data model. By the end you will understand how each layer works, what it catches, and how to run the full suite as both a CLI tool and a pytest test suite with HTML reporting. This is the gate that stands between a YAML change and your production network.

What We Built

Lab 1 gave us a data model and a basic validation script. Lab 2 replaces that script with a structured validation framework that runs four distinct layers of checks, each one building on the previous. The framework produces machine-readable results with rule IDs so that a CI pipeline can gate on specific checks, and it integrates with pytest so that every validation rule is also a test case with proper reporting.

The four layers are format, syntax, semantic, and compliance. They run in order from cheapest to most expensive. If a file cannot be parsed as YAML, there is no point running Pydantic schema validation against it. If the schema says a field is the wrong type, there is no point checking whether it references a VRF that exists in another file. Each layer assumes the previous layer passed.

Prerequisites

You need a working Lab 1 setup: the project directory with all YAML data files, Pydantic schemas, and Python dependencies installed via `uv sync`. If you can run `uv run python validate.py` from the old Lab 1 code and see it pass, you are ready.

Part 1: The Validators Package

The validation logic lives in `validators/`, a Python package with one module per layer plus a shared `__init__.py` that defines the common types.

Common Types

```
cat validators/__init__.py
```

Two types anchor the entire framework. `ValidationLevel` is an enum with four values: FORMAT, SYNTAX, SEMANTIC, and COMPLIANCE. `ValidationResult` is a frozen dataclass that carries the level, a rule ID, a human-readable message, a pass/fail boolean, an optional file path, and an optional list of detail strings for when a single check has multiple things to report.

The rule ID is the key design decision. Every check gets a stable identifier like SEM-03 or CMP-01. This means a CI pipeline can say “block the merge if any SEM rule fails” without

parsing English error messages. It also means an engineer reading a failure report can grep the codebase for that rule ID and land directly on the code that flagged the issue.

The `__init__.py` also contains `parse_all_files()`, a helper that loads every YAML file and runs it through its Pydantic model in one call. This is shared between the CLI entry point and the pytest fixtures so there is exactly one code path for loading data.

File Specs

The `FILE_SPECS` list maps each component name to its relative file path, Pydantic model class, and optional root key. This is the single place where the framework knows "fabric lives in data/fabric.yaml and validates against FabricConfig, and the data is nested under the `fabric` key." Adding a new data file to the model means adding one tuple to this list.

Part 2: Layer 1 – Format Validation

```
cat validators/format_validator.py
```

Format validation answers four questions per file.

FMT-01 checks whether the file exists at all. A missing file is an immediate failure. This catches the scenario where someone renames a YAML file or forgets to commit it.

FMT-02 checks whether the file is valid YAML. A misplaced colon, an unclosed bracket, or a tab character where YAML expects spaces all produce parse errors that prevent any further processing.

FMT-03 checks whether the parsed YAML produces a mapping (a Python dict). YAML can also produce lists or scalars at the top level, and our data model always expects a mapping.

FMT-04 checks whether the required top-level key is present. `fabric.yaml` nests its data under a `fabric:` key, and `defaults.yaml` nests under `defaults:`. If someone accidentally deletes that top-level key while editing, FMT-04 catches it before the data reaches Pydantic.

Run It

```
uv run python -c "
from pathlib import Path
from validators.format_validator import validate_format
for r in validate_format(Path('.')):
    status = 'PASS' if r.passed else 'FAIL'
    print(f'  {status}  {r.rule_id}  {r.message}')
"
```

You should see eight PASS results, one per data file.

Part 3: Layer 2 – Syntax Validation

```
cat validators/syntax_validator.py
```

Syntax validation feeds each YAML file through its Pydantic model. This is where the Lab 1 schemas earn their keep. Every `Field(ge=1, le=4094)` constraint, every `@field_validator`, every `@model_validator(mode="after")` in `schemas/models.py` runs during this layer.

The syntax validator produces one SYN-01 result per file. If a file has multiple schema violations, each one becomes a separate FAIL result with the field path and Pydantic's error message. A file that passes gets a single PASS.

This layer catches things like an ASN of zero, a device name with uppercase letters or special characters, a VLAN ID of 9999, an OSPF dead interval shorter than the hello interval, or an access-mode interface with multiple VLANs assigned. These are all structurally valid YAML, which is why format validation passes them. But they violate the schema constraints that define what a correct data model looks like.

Run It

```
uv run python -c "
from pathlib import Path
from validators.syntax_validator import validate_syntax
for r in validate_syntax(Path('.')):
    status = 'PASS' if r.passed else 'FAIL'
    print(f'  {status}  {r.rule_id}  {r.message}')
"
```

Eight PASS results, one per file.

Part 4: Layer 3 – Semantic Validation

```
cat validators/semantic_validator.py
```

This is the largest and most important layer. Format validation proves each file is readable. Syntax validation proves each file is individually correct. Semantic validation proves the files are correct together. It operates on the validated Pydantic model instances, not on raw YAML, so it can traverse the object graph directly.

There are 16 semantic rules. Here is what each one checks and why it matters.

Referential Integrity (SEM-01 through SEM-07)

SEM-01 verifies that every device named in a P2P link actually exists in topology.yaml. A typo like `spine3` in a link definition would pass format and syntax validation because "spine3" is a perfectly valid string. But it references a device that does not exist, which means the link cannot be wired.

SEM-02 checks that every device listed as a route reflector in overlay.yaml exists in the topology and has the spine role. A leaf acting as a route reflector is a design violation in a spine-leaf fabric.

SEM-03 ensures consistency between the two places where route reflectors are declared. The topology file has a `route_reflector: true` flag per device. The overlay file has a `route_reflectors` list. These must agree. If spine1 is flagged as an RR in the topology but missing from the overlay's list, something is out of sync.

SEM-04 checks that every VRF referenced by a network segment in networks.yaml is actually defined in vrfs.yaml. Referencing a VRF named STAGING when only PRODUCTION, DEVELOPMENT, and MANAGEMENT exist means the network segment has nowhere to live.

SEM-05 does the same check for interface assignments: every device named in interfaces.yaml must exist in the topology.

SEM-06 verifies that every VLAN ID used in an interface assignment corresponds to a network segment. Assigning VLAN 50 to a port when no network defines VLAN 50 means traffic on that port has no L3 gateway and no VXLAN mapping.

SEM-07 confirms that host-facing interfaces are only assigned to leaf and border devices. Spines are fabric-only in a spine-leaf design. They do not connect to servers or firewalls.

IP Address Range Checks (SEM-08 through SEM-10)

SEM-08, SEM-09, and SEM-10 verify that loopback addresses, P2P link addresses, and management addresses all fall within the ranges declared in fabric.yaml. The fabric defines three non-overlapping IP ranges. A loopback address that wanders into the P2P range, or a management IP outside the management range, indicates either a typo or a misunderstanding of the addressing plan.

Cross-Domain Consistency (SEM-11 through SEM-12)

SEM-11 checks that L2 VNIs (assigned to network segments) and L3 VNIs (assigned to VRFs) do not collide. VNIs must be globally unique across the fabric. A collision means two different things share the same VXLAN tunnel identifier, which causes traffic to land in the wrong place.

SEM-12 enforces that every device's ASN matches the fabric ASN. This is an iBGP design where all devices share ASN 65000. A device with a different ASN would form an eBGP peering instead, which is a fundamentally different routing relationship.

Fabric Design Rules (SEM-13 through SEM-16)

SEM-13 verifies the full mesh: every spine must have a direct link to every leaf and every border. A missing link means a leaf has no path through that spine, breaking the redundancy model.

SEM-14 checks that no spine-to-spine links exist. In a spine-leaf design, spines do not peer with each other directly. Traffic between spines always transits through a leaf. A spine-to-spine link suggests someone is building a different topology than the one declared in the data model.

SEM-15 verifies uplink redundancy: every leaf and border device must connect to at least two spines. A single-uplink leaf is a single point of failure. If that one spine goes down, the leaf is isolated from the fabric.

SEM-16 checks that every defined VRF has at least one network segment. A VRF with no networks is dead weight. It consumes configuration on every device but carries no traffic. This usually means someone defined the VRF but forgot to add networks, or deleted the networks without cleaning up the VRF.

Run It

```
uv run python -c "
from pathlib import Path
```

```
from validators import parse_all_files
from validators.semantic_validator import validate_semantic
parsed, _ = parse_all_files(Path('.'))
for r in validate_semantic(parsed):
    status = 'PASS' if r.passed else 'FAIL'
    print(f'  {status}  {r.rule_id}  {r.message}')
"
```

Sixteen PASS results.

Part 5: Layer 4 – Compliance Validation

```
cat validators/compliance_validator.py
```

Compliance validation is where organizational policy lives as code. Everything that passed the first three layers is technically correct and logically consistent. Compliance checks whether it also meets your standards.

The difference between semantic and compliance is intent. A semantic failure means “this will not work.” A compliance failure means “this will work but it violates our operational standards.” Both block the pipeline, but they communicate different things to the engineer reviewing the failure.

There are 7 compliance rules.

CMP-01 requires jumbo MTU (>= 9000) on fabric links. Jumbo frames are standard practice on data center fabrics because VXLAN adds 50 bytes of encapsulation overhead. Running 1500-byte MTU on the fabric means VXLAN-encapsulated frames get fragmented, which destroys performance.

CMP-02 requires /31 subnets on all point-to-point links. Using /30s wastes an address per link. Using /24s is worse. RFC 3021 established /31s for point-to-point links in 2000 and every modern NOS supports them.

CMP-03 checks that the description prefix matches the organizational standard, which is “NaC-Managed.” Every interface and object managed by this automation framework carries this prefix so that operators can instantly tell whether a configuration element was placed by automation or by hand.

CMP-04 requires ARP suppression to be enabled. ARP suppression reduces broadcast traffic in VXLAN overlays by having the VTEP answer ARP requests locally instead of flooding them across the fabric. Disabling it in a modern EVPN deployment is almost never intentional.

CMP-05 checks that each route reflector’s cluster ID matches its loopback address. This is a common convention that simplifies troubleshooting. When you see a cluster ID in a BGP update, you immediately know which physical device originated it.

CMP-06 verifies that the default BGP hold time is at least 3x the keepalive interval. This is an RFC 4271 requirement. Violating it means BGP sessions will flap under normal keepalive jitter.

CMP-07 requires management MTU to be exactly 1500. Management traffic does not traverse the VXLAN overlay, so jumbo frames are unnecessary and some management tools do not handle them correctly.

Run It

```
uv run python -c "
from pathlib import Path
from validators import parse_all_files
from validators.compliance_validator import validate_compliance
parsed, _ = parse_all_files(Path('.'))
for r in validate_compliance(parsed):
    status = 'PASS' if r.passed else 'FAIL'
    print(f'  {status}  {r.rule_id}  {r.message}')
"
```

Seven PASS results.

Part 6: The Full Pipeline

The `validate.py` script at the project root runs all four layers in sequence. It stops at the first layer that has any failures, because there is no value in reporting semantic errors when the YAML cannot even be parsed.

```
uv run python validate.py
```

You should see 39 checks across four layers, all passing, followed by a summary table of the fabric data model.

HTML Report Generation

For CI pipelines and documentation, the validator can produce an HTML report through pytest:

```
uv run python validate.py --html reports/validation-report.html
```

This delegates to pytest with the `pytest-html` plugin and produces a self-contained HTML file you can open in a browser or attach to a pull request.

Part 7: The pytest Test Suite

The `tests/` directory contains four test modules, one per validation layer, plus a `conftest.py` with shared fixtures.

```
uv run pytest -v
```

You should see 27 tests pass. The test suite covers two dimensions for each layer.

The first dimension is positive testing: run the validators against the current data model and assert that everything passes. This is the “our data model is correct” confidence check. If any of these fail, something in the data files is broken.

The second dimension is negative testing: construct intentionally broken data and assert that the correct rule catches it. This is the “our validators actually work” confidence check. Without negative tests, you could delete every validation rule and the positive tests would still pass because there is nothing wrong with the data.

What the Negative Tests Cover

The format tests create corrupt YAML files in a temp directory and verify that FMT-01, FMT-02, and FMT-04 fire correctly.

The syntax tests patch individual YAML values to violate schema constraints. An ASN of zero, a device name with uppercase letters, an OSPF dead interval shorter than hello, a VLAN ID above 4094. Each one targets a specific Pydantic validator.

The semantic tests use `deepcopy` to create modified versions of the parsed model objects. They add a link to a nonexistent device, a network referencing an undefined VRF, an interface assignment on a spine, a removed link that breaks the full mesh, a spine-to-spine link, and a VRF with its networks stripped out. Six different rules, six different failures.

The compliance tests modify defaults and overlay config: drop the MTU below 9000, widen a /31 to a /30, change the description prefix, disable ARP suppression, set a wrong cluster ID, and set a non-standard management MTU. Six rule violations across seven rules.

Running Individual Test Modules

If you only want to see one layer:

```
uv run pytest tests/test_format.py -v
uv run pytest tests/test_syntax.py -v
uv run pytest tests/test_semantic.py -v
uv run pytest tests/test_compliance.py -v
```

HTML Test Report

```
uv run pytest --html=reports/validation-report.html --self-
        contained-html -v
```

This produces a detailed HTML report showing each test case, its result, and timing. The `--self-contained-html` flag embeds all CSS and JavaScript into a single file so it can be shared without a web server.

Part 8: Break Something on Purpose

This is the section that makes the framework real. Run through each example, see the failure, then revert it.

Example 1: Overlapping Subnets (Caught by Syntax Layer)

Edit `data/services/networks.yaml` and change the dev-general subnet to overlap with prod-web:

```
  - name: dev-general
    vni: 20010
    vlan_id: 110
    subnet: 10.100.10.0/24
    gateway: 10.100.10.1
```

```
    vrf: DEVELOPMENT
    description: "Development general purpose"
```

```
uv run python validate.py
```

The syntax layer catches it: “Subnet overlap: ‘prod-web’ (10.100.10.0/24) overlaps with ‘dev-general’ (10.100.10.0/24).” The semantic and compliance layers never run because the pipeline stops at the first failure.

Revert the change before continuing.

Example 2: Undefined VRF Reference (Caught by Semantic Layer)

Edit `data/services/networks.yaml` and change a network’s VRF to something that does not exist:

```
    vrf: STAGING
```

```
uv run python validate.py
```

Format and syntax pass. The semantic layer catches it at SEM-04: “Networks reference undefined VRFs” with the detail “Network ‘prod-web’ references VRF ‘STAGING’.”

Revert the change.

Example 3: Non-Jumbo MTU (Caught by Compliance Layer)

Edit `data/defaults.yaml` and change the fabric link MTU:

```
  fabric_link_mtu: 1500
```

```
uv run python validate.py
```

All three earlier layers pass. The compliance layer catches it at CMP-01: “Fabric link MTU is 1500, policy requires >= 9000.”

Revert the change.

Example 4: Spine-to-Spine Link (Caught by Semantic Layer)

Add a new link to `data/underlay.yaml`:

```
  - name: spine1-to-spine2
    a_device: spine1
    a_interface: eth99
    a_ip: 10.0.1.200/31
    b_device: spine2
    b_interface: eth99
    b_ip: 10.0.1.201/31
    link_type: point_to_point
```

```
uv run python validate.py
```

SEM-14 catches it: “Spine-to-spine links are not allowed in spine-leaf design.”

Revert the change.

Part 9: Commit the Validation Framework

Make sure all your break-it demos are reverted and validation passes cleanly:

```
uv run python validate.py
uv run pytest -v
```

All 39 validation checks and 27 tests should pass. Now check what is new since the Lab 1 commit:

```
git status
```

You should see the new `validators/` package, `tests/` directory, updated `validate.py`, updated `pyproject.toml` and `uv.lock` (with pytest and pytest-html added), and the Lab 2 build guide. Stage and commit:

```
git add validators/ tests/ validate.py pyproject.toml uv.lock
        docs/lab2-build-guide.md
git status
```

Review the staged files. The validators and tests are the core of this lab. The updated `validate.py` now delegates to the four-layer framework. The `pyproject.toml` has the new test dependencies pinned.

```
git commit -m
        "Lab 2: four-layer validation framework with pytest
        suite"
```

Check the log:

```
git log --oneline
```

Two commits now. The first established the data model. The second added the validation framework that protects it. This is the pattern going forward: build something, verify it works, commit it. Every lab ends with a commit that captures the work in a reviewable, revertable snapshot.

Part 10: What We Proved

By the end of this lab you have demonstrated four things.

First, that validation is not a single check but a layered process. Each layer has a distinct purpose and operates at a different level of abstraction. Mixing them together makes errors harder to diagnose. Separating them means the engineer sees “your YAML is broken” or “your VRF reference is wrong” or “your MTU violates policy” instead of a generic “validation failed.”

Second, that every rule has a stable ID and a clear message. SEM-04 always means “network references an undefined VRF.” CMP-01 always means “fabric MTU below policy.” These IDs are

the contract between the validation framework and the CI pipeline. When Lab 4 wires this into GitHub Actions, the pipeline will gate on these IDs.

Third, that the validators are themselves tested. The pytest suite does not just run the validators against good data. It runs them against bad data and asserts they catch the right thing. This is the difference between "we have validators" and "we have validators we can trust." When someone adds a new rule, they also add a test that proves it works.

Fourth, that validation catches errors at the right layer. A subnet overlap is a Pydantic schema constraint, so it fires in the syntax layer before semantic validation even runs. A missing VRF reference spans two files, so it fires in the semantic layer. A policy violation on MTU is an organizational standard, not a logical error, so it fires in the compliance layer. Each failure lands in the layer that makes sense, which tells the engineer not just what is wrong but what kind of wrong it is.

Troubleshooting

pytest cannot find the tests directory: Make sure you are running from the project root (the directory that contains `pyproject.toml`). The `[tool.pytest.ini_options]` section in `pyproject.toml` sets `testpaths = ["tests"]`.

Import errors when running validators directly: The validators package imports from `schemas.models`, which requires the project root on the Python path. Using `uv run` handles this automatically. If you are running Python directly, make sure your working directory is the project root.

HTML report is empty or missing: Check that `pytest-html` is installed. Run `uv sync` to ensure all dependencies are present. The `--self-contained-html` flag requires pytest-html 4.0+, which is pinned in the project dependencies.

A compliance rule fails unexpectedly: Compliance rules encode specific policy values. If your organization uses different standards (1500 MTU on fabric links, /30 P2P subnets, a different description prefix), update the constants in `validators/compliance_validator.py` to match your policies. The rules are intentionally simple Python functions that are easy to modify.

Semantic validation reports missing components: This means one or more YAML files failed format or syntax validation. Run the format and syntax layers first (`validate.py` does this automatically) and fix those errors before semantic validation can run.